

Анализ графов

Пентегова Мария Сергеевна

Группа: 25.Б41-мм

Кафедра: *системного программирования*

Научный руководитель: *Григорьев Семён Вячеславович*

Номер семестра практики: 2

1. Введение

BFS (Breadth-First Search, поиск в ширину) — фундаментальный алгоритм обработки графов, позволяющий находить расстояния от источника до достижимых вершин в невзвешенном графе. Для больших разреженных графов существенное значение имеет эффективное представление данных и реализация обхода. В работе сравниваются классический BFS на основе очереди и CSR с матричной реализацией на базе SuiteSparse:GraphBLAS.

2. Постановка задачи

Цель работы — сравнить производительность классической и GraphBLAS-реализаций BFS на графах различных типов.

Задачи:

1. реализовать BFS для одного и нескольких источников;
2. реализовать чтение `.mtx`, преобразование в CSR и поиск вершины с максимальной степенью;
3. провести эксперименты на реальных графах SuiteSparse;
4. проанализировать зависимость времени выполнения от структуры графа.

3. Реализация

Программа на C11 состоит из следующих компонентов: `utils.c/.h` — чтение графов, списки смежности, CSR; `classic_bfs.c` — классический BFS; `graphblas_bfs.c` — BFS на основе GraphBLAS; `main.c`, `benchmark.c` — запуск и измерение времени.

Для классического BFS используется формат CSR (Compressed Sparse Row). Список смежности вершины v хранится в диапазоне

$$[\text{row_ptr}[v], \text{row_ptr}[v + 1]).$$

Обход имеет сложность $O(|V| + |E|)$, а обработка соседей вершины — $O(\deg(v))$.

Матричная реализация использует SuiteSparse:GraphBLAS — реализацию C API GraphBLAS для операций над разреженными матрицами.

Используемые технологии: C11, CMake 3.10+, SuiteSparse:GraphBLAS, CTest, `assert.h`. Время измеряется функцией `clock_gettime()` с `CLOCK_MONOTONIC`, что соответствует POSIX-интерфейсу.

4. Эксперименты

Эксперименты проведены на 7 реальных графах из SuiteSparse Matrix Collection. Для однопоточного эксперимента с одним источником выбиралась вершина с максимальной степенью. Для multisource использовались четыре равномерно распределённых источника.

Каждая реализация запускалась 10 раз. Для каждого набора измерялись среднее, минимальное и максимальное время. Перед измерениями выполнялся дополнительный прогревочный запуск.

Граф	CP	CM	GB-L	GB-M	CP/GB-L
com-Orkut	2652628	2700768	486500	420307	5.45x
soc-LiveJournal1	1915627	1976468	358255	331458	5.35x
road_usa	1451387	1588466	25839879	22121848	0.06x
roadNet-CA	127543	110558	663630	670218	0.19x
web-Google	100233	80683	101149	106972	0.99x
cit-Patents	17500	4730	181639	46757	0.10x
com-Amazon	38967	31974	61923	62144	0.63x

Таблица 1: Среднее время BFS по 10 запускам, мкс. CP — Classic Parent, CM — Classic Multi, GB-L — GraphBLAS Level, GB-M — GraphBLAS Multi.

4.1. Анализ результатов

На `com-Orkut` и `soc-LiveJournal1` GraphBLAS показал меньшее среднее время: 486 500 и 358 255 мкс против 2 652 628 и 1 915 627 мкс у Classic Parent.

На дорожных графах результат противоположный. Для `road_usa` классический BFS выполнялся за 1 451 387 мкс против 25 839 879 мкс у GraphBLAS, а для `roadNet-CA` — 127 543 против 663 630 мкс.

На `web-Google` времена близки: 100 233 и 101 149 мкс. На `cit-Patents` и `com-Amazon` классическая реализация также быстрее. В частности, для `cit-Patents` получено 17 500 против 181 639 мкс.

Таким образом, на исследованной выборке GraphBLAS быстрее на двух графах, классический BFS — на четырёх, а на одном результаты близки. Это указывает на зависимость производительности от структуры графа, а не только от числа вершин и рёбер. Для анализа влияния ширины фронта BFS необходимо дополнительно измерять число вершин на каждом уровне.

5. Тестирование и заключение

Корректность проверяется с помощью CTest и `assert.h` на тестовом графе из 5 вершин. Проверяются посещение вершин, расстояния, родители, результаты multisource-обхода и поэлементное соответствие классической и GraphBLAS-реализаций. Все тесты проходят успешно (100% passed).

В работе реализованы и сравнены два подхода к BFS: классический обход с очередью и CSR и матричный подход на основе SuiteSparse:GraphBLAS. Эксперименты на семи реальных графах показали, что производительность зависит от структуры графа. GraphBLAS оказался быстрее на двух социальных графах, классическая реализация — на четырёх графах, а на `web-Google` результаты были близки. Ориентированность графа сама по себе не объясняет различия производительности.

Полученные результаты относятся к использованному набору графов и условиям эксперимента. Для более детального исследования необходимо учитывать характеристики фронта BFS на каждом уровне.

Репозиторий: https://github.com/MariaPentegova/graph_blas_bfs-2

6. Список использованных источников

1. IEEE Std 1003.1-2017. *POSIX.1-2017: System Interfaces*. The Open Group. <https://pubs.opengroup.org/onlinepubs/9699919799/>.
2. GHOST966. *Алгоритмы на графах: BFS — поиск в ширину*. Хабр, 2025. <https://habr.com/ru/articles/969450/>.

3. Intel Corporation. *Intel oneMKL: Sparse Storage Formats*. 2025. <https://www.intel.com/content/www/us/en/docs/onemkl/developer-reference-dpcpp/2025-2/sparse-storage-formats.html>.
4. GraphBLAS Forum. *The GraphBLAS C API Specification*. Version 2.1, 2023. https://graphblas.org/docs/GraphBLAS_API_C_v2.1.0.pdf.
5. Davis T. A. *SuiteSparse:GraphBLAS*. ACM Transactions on Mathematical Software, 2023. <https://github.com/DrTimothyAldenDavis/GraphBLAS>.
6. *SuiteSparse Matrix Collection*. <https://sparse.tamu.edu/>.